

EduGenie

Google Gemini Powered Learning Assistant

Phase 2: Requirement Analysis

1. Introduction

Requirement Analysis defines what the EduGenie system should provide, who will use it, what problems it should solve, and what technical and functional requirements are necessary to develop the system.

EduGenie is a lightweight AI-powered learning assistant designed to simplify learning through generative AI. The system provides educational support through question answering, simplified explanations, quiz generation, summarization, and personalized learning recommendations.

2. Project Overview

EduGenie is designed to assist students in understanding and learning educational topics more effectively.

The system uses:

- **FastAPI** for the backend
- **HTML, CSS, and JavaScript** for the frontend
- **Gemini 1.5 Pro** through an API for Q&A, summarization, quiz generation, and learning recommendations
- **LaMini-Flan-T5-783M** local model for concept explanation

The system provides five main learning functions:

1. Explain
2. QnA
3. Quiz
4. Summary
5. Recommend Path

3. Problem Requirements

The system is intended to address common learning requirements such as:

- Students needing simple explanations of difficult concepts
- Students needing quick answers to educational questions
- Students needing summaries of longer learning content
- Students needing practice questions for self-assessment
- Students needing guidance on how to learn a topic step by step

The system should provide these functions through a simple and accessible interface.

4. Target Users

Primary Users

Students

Students can use EduGenie to:

- Understand difficult concepts
- Ask educational questions
- Generate quizzes
- Summarize learning material
- Receive learning recommendations

Secondary Users

The system may also be useful as an educational support tool for learners who need additional assistance with understanding study topics.

5. Functional Requirements

The following functional requirements define the capabilities that EduGenie must provide:

FR1 - Question and Answer

The system shall allow users to enter an educational question and receive a concise answer.

FR2 - Concept Explanation

The system shall allow users to enter a topic or concept and receive a simplified explanation.

FR3 - Quiz Generation

The system shall generate quiz questions based on the user's selected topic or input. The current project generates three questions with four options each and provides answer checking.

FR4 - Summarization

The system shall allow users to provide content and receive a concise summary.

FR5 - Learning Recommendations

The system shall provide learning recommendations and stepwise guidance for a selected topic.

FR6 - User Input

The frontend shall provide a text area or input mechanism through which users can enter their learning request.

FR7 - Task Selection

The frontend shall allow users to select the required learning task, including:

- Explain
- QnA
- Quiz
- Summary
- Recommend Path

FR8 - Result Display

The system shall display the generated response or result to the user through the web interface.

6. Non-Functional Requirements

6.1 Usability

The interface should be simple and easy for students to understand and use without requiring extensive training or documentation.

6.2 Performance

The application should provide responses efficiently while interacting with the configured AI models and APIs.

6.3 Reliability

The system should handle user requests and return appropriate results without unnecessary failures.

6.4 Maintainability

The project should be organized into separate modules for different functions such as Q&A, explanation, quiz generation, summarization, and learning recommendations.

6.5 Security

The Gemini API key must be kept confidential and should not be committed to the GitHub repository. Environment variables or appropriate secret-management methods should be used for sensitive credentials.

6.6 Compatibility

The web application should be usable through a modern web browser.

7. Technical Requirements

Software Requirements

- Python
- FastAPI
- HTML
- CSS
- JavaScript
- Gemini API
- LaMini-Flan-T5-783M local model
- Uvicorn
- Required Python dependencies listed in `requirements.txt`

Development Environment

The project can be run locally using:

```
uvicorn main:app --reload
```

The application can then be accessed through:

```
http://127.0.0.1:8000
```

8. System Requirements

The system requires:

- A computer capable of running the Python application and required local model
- Python environment with the required dependencies
- Internet connectivity for Gemini API-based functions
- A web browser for accessing the frontend
- A valid Gemini API key configured through environment variables

9. Input Requirements

The system should accept user-provided educational input such as:

- Questions
- Topics
- Concepts
- Learning content for summarization

The input is processed according to the selected task.

10. Output Requirements

Depending on the selected function, EduGenie should provide:

Function	Expected Output
QnA	Concise answer to the question
Explain	Simplified explanation of the concept
Quiz	Three questions with four options each and answer checking
Summary	Concise summary of the provided content
Recommend Path	Learning resources and stepwise guidance

11. API Requirements

The backend should provide the following endpoints:

- /qa – Handle question and answer requests
- /explain – Handle concept explanation requests
- /quiz – Handle quiz generation requests
- /summarize – Handle content summarization requests
- /learn/recommendations – Handle learning recommendation requests

Each endpoint should process the relevant request and return the corresponding learning result.

12. Security Requirements

The project should follow basic security practices:

- Keep the Gemini API key private
- Store sensitive credentials using environment variables
- Use `.env.example` as a template without exposing actual secrets
- Never commit real API keys to GitHub

13. Constraints

The project has the following known constraints:

- AI-generated responses may require verification for accuracy
- The local explanation model may require sufficient device resources
- Personalization and general usability depend on the quality of user input and prompts
- Prompt tuning may be required to improve generated responses
- The user interface should remain simple and usable

14. Future Requirements / Possible Enhancements

⚠ Future Enhancements (Not Currently Implemented)

The following are enhancements planned for future phases and **must not be presented as current implemented features**:

- Voice interaction
- Multilingual support
- Mobile or offline features
- Learning progress dashboards
- Gamification
- Adaptive learning paths
- Group study features
- Teacher and parent dashboards
- Learning Management System (LMS) integration
- Image and PDF input processing
- Real-time synchronization across devices
- Notifications and reminders

15. Requirement Summary

The requirement analysis establishes that EduGenie should provide an easy-to-use AI-powered learning interface with five core capabilities:

1. Question and Answer
2. Simplified Explanation
3. Quiz Generation
4. Summarization
5. Learning Recommendations

The system requires a FastAPI backend, web frontend, Gemini API integration, and a local explanation model. Security, usability, maintainability, and reliable handling of user requests are important non-functional requirements.

Functional requirements define what the system must do, while non-functional requirements define how well it should perform its functions. Technical requirements specify the tools and technologies required for implementation.

16. Conclusion

The Requirement Analysis phase defines the functional, non-functional, technical, security, input, and output requirements of EduGenie. These requirements provide the foundation for the next phases of project design, planning, development, testing, documentation, and demonstration.

This Phase 2 document serves as the specification baseline against which all subsequent project decisions, design choices, and implementation work will be evaluated. Clear documentation of requirements ensures that the development team understands what needs to be built and enables stakeholders to verify that the final product meets the intended objectives.

The requirements documented here address core learning support needs while maintaining simplicity and accessibility as core design principles. Future phases will build upon this requirement specification to design and implement the complete EduGenie learning assistant system.